CryptoLens

Phase 3: Final Architecture & FinOps Report

A Medallion Lakehouse for Binance market data: Bronze, Silver, Gold and Power BI

Course	Data Analysis and Visualization (semester project)
Team	Abdullah Saeed (24L-2529), Arqum Umais (24L-2591)
Platform	Databricks Free Edition (serverless), Delta Lake, Unity Catalog, Power BI Desktop
Repository	https://github.com/x-abdullah-saeed-x/CryptoLens
Date	24 October 2026

1. Executive summary

CryptoLens is an automated lakehouse pipeline for 1-minute candlestick (OHLCV) data of the 50 most actively traded Binance USDT spot pairs. Raw files are downloaded and checksum-verified on a local machine, landed in a Unity Catalog Volume, and processed through Bronze (raw, explicitly typed), Silver (cleaned, deduplicated) and Gold (a star schema built for analytics). A Databricks Job chains the three stages so that Gold refreshes automatically after Silver, every step is written to a shared audit table, and a four-page Power BI report reads the Gold layer.

Item	Result
Scope	Top-50 USDT pairs by 24h quote volume, 1-minute interval
Full load	49 symbols for August 2026 (HYPEUSDT has no monthly file), 2,187,360 candles, about 289 MB of CSV
Incremental load	50 symbols for 25 September 2026, 72,000 candles, about 9.7 MB
Bronze	2,259,360 candles (full + incremental), explicit schema, idempotent MERGE
Gold row counts	dim_symbol 765; dim_date 56; fact_daily_ohlcv 1,569; volatility_daily 1,569; moving_averages 1,569; symbol_correlation 1,890; 0 fact rows without a matching dim_symbol
Automation	Job cryptolens_daily (bronze, then silver, then gold), measured 14 min 41 s end to end
Dashboard	4 pages, 8 active many-to-one relationships matching the star schema
Main engineering finding	Binance 2026 files use microsecond timestamps; the pipeline assumed milliseconds (section 6.3)

Requirement traceability

Requirement	Where in this report / repo	Evidence
1. Star schema (2 dimensions, 4 fact tables)	Section 3.1; notebooks/03_gold_transform	dim_symbol, dim_date; fact_daily_ohlcv, volatility_daily, moving_averages, symbol_correlation; shared integer keys
1. ER diagram / Gold data dictionary	Section 3.2, Figure 2; docs/gold_data_dictionary.md	Primary and foreign keys, grain, load strategy, column definitions
1. Read optimisation	Section 3.3; 03_gold_transform step 9	OPTIMIZE with ZORDER BY on all four fact tables, each step logged
2. End-to-end automation	Section 4.1, Figure 3, section 4.2	Job cryptolens_daily: bronze, silver, gold; MERGE on the fact table, idempotent re-runs
2. Logging continuation	Section 4.3	layer = Silver-to-Gold rows with source rows read, start/end time, rows inserted/updated
3. BI connection	Section 5.1	Power BI Databricks connector to a serverless SQL warehouse (Import mode)
3. Dashboard and audit visuals	Sections 5.3 and 5.4	Four pages including Pipeline Audit; KPIs, time series, heatmap, categorical splits
3. Semantic layer	Section 5.2, Figure 5	Eight active many-to-one relationships matching the star schema
4. Architecture diagram	Section 2, Figure 1	Raw source to Bronze, Silver, Gold and Power BI
4. Decision log	Section 6	Partitioning (6.2), schema drift (6.3), PII (6.4)

Requirement	Where in this report / repo	Evidence
4. Spark UI and DAG analysis	Section 7, Figure 6	Plan graph with shuffles, broadcast join, predicate pushdown explained
4. Cost and FinOps	Section 8	Measured 14 min 41 s run; daily vs hourly annual cost model; optimisations

2. Architecture

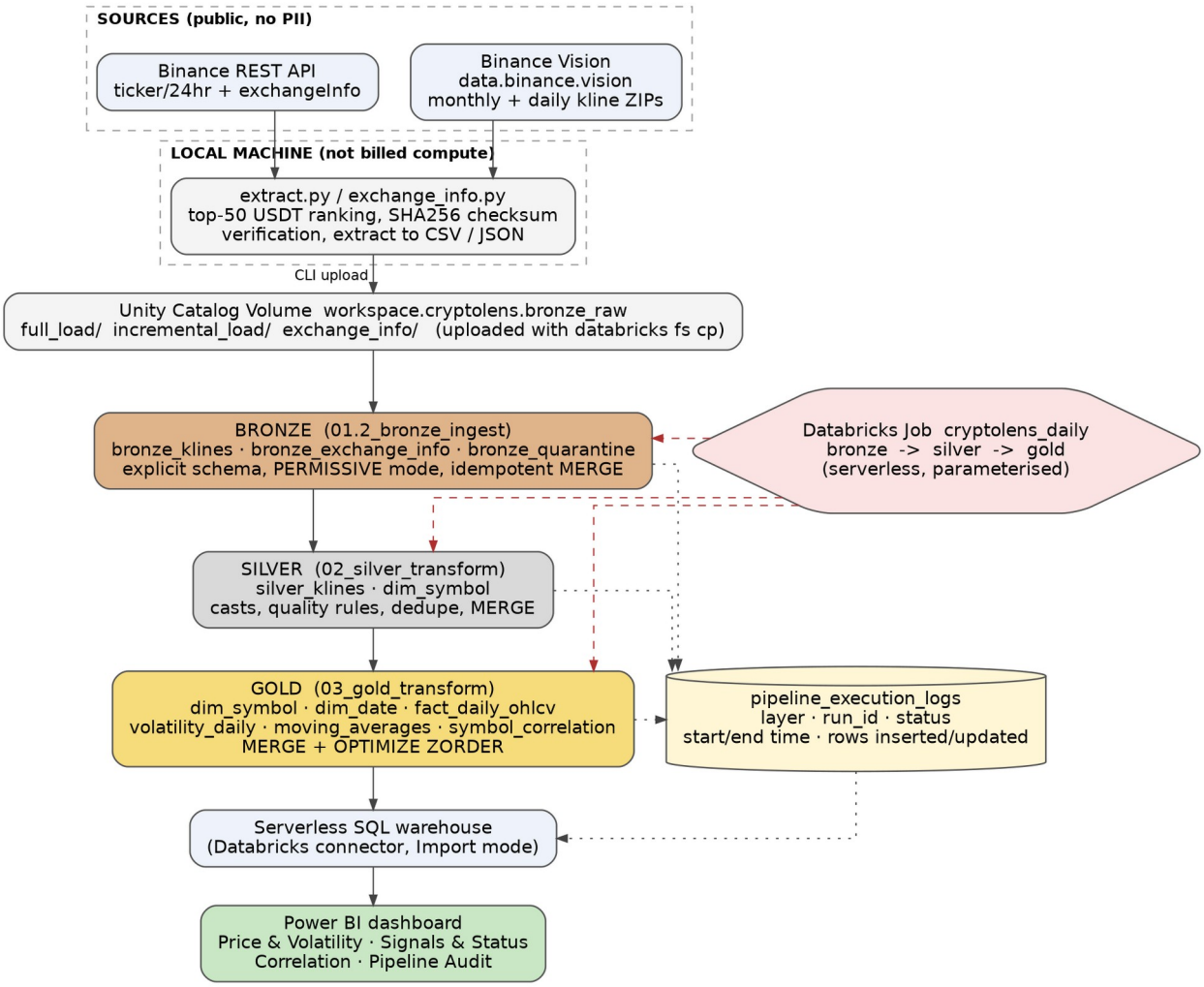


Figure 1. End-to-end data flow from the Binance sources to Power BI, with orchestration and audit logging.

Data moves in one direction. The two Binance sources are queried from a local machine, because Databricks serverless compute could not reach the Binance endpoints (the REST call returned HTTP 451 and the Phase 1 DNS test could not resolve data.binance.vision). The local scripts rank the top-50 USDT pairs, download one monthly file per symbol (full load) and one daily file per symbol (incremental load), verify each ZIP against Binance's own SHA256 sidecar file, and write CSV and JSON files. Only verified files are uploaded to the Volume with the Databricks CLI, so the heavy download never uses billed compute.

Layer	Objects	Role
Bronze	bronze_klines, bronze_exchange_info, bronze_quarantine	Raw rows read with an explicit StructType in PERMISSIVE mode; rows that fail parsing go to the quarantine table; MERGE on (symbol, interval, load_type, open_time).

Layer	Objects	Role
Silver	silver_klines, dim_symbol	Epoch values converted to timestamps, prices cast to DECIMAL(18,8) and volumes to DECIMAL(28,8), quality rules (high >= low, open and close inside the range), de-duplication, MERGE on (symbol, interval, open_time).
Gold	Two dimensions and four fact tables in workspace.gold	Daily roll-up of the 1-minute candles, derived indicators, MERGE for the main fact table, OPTIMIZE with Z-ORDER.
Audit	pipeline_execution_logs	One row per file or step at every layer: status, start and end time, rows inserted and updated, error message.
BI	Serverless SQL warehouse and Power BI	Power BI imports the Gold tables and the audit table through the Databricks connector.

3. Gold layer dimensional model

3.1 Star schema design

The Gold layer is a star schema, not a copy of Silver. Two conformed dimensions describe who (the trading pair) and when (the calendar day). Four fact tables hold the measures at different grains of the same two dimensions. Every fact table carries the same integer surrogate keys, so a single symbol slicer and a single date slicer in Power BI filter all of them.

Table	Type	Grain (one row =)	Primary key
dim_symbol	Dimension	one USDT trading pair	symbol_key
dim_date	Dimension	one calendar day	date_key (yyyyMMdd)
fact_daily_ohlcv	Fact	symbol x day	(symbol_key, date_key)
volatility_daily	Derived fact	symbol x day	(symbol_key, date_key)
moving_averages	Derived fact	symbol x day	(symbol_key, date_key)
symbol_correlation	Derived fact	symbol pair x day	(symbol_a_key, symbol_b_key, date_key)

Why a star and not a snowflake. Each dimension is already flat: a trading pair has a handful of descriptive attributes and a date has calendar parts. Normalising them further would add joins and no value, and the BI tool would have to traverse extra relationships.

Keys. symbol_key is abs(xxhash64(symbol)), computed identically in every table. It is deterministic, needs no lookup against the dimension during the fact load, and is reproducible if a table is rebuilt. date_key is the integer yyyyMMdd. The check "fact rows with no dim_symbol match" returned 0.

Slowly changing attributes. dim_symbol records status changes (previous_status, status_changed_date) and marks a pair DELISTED when it disappears from the latest exchangeInfo snapshot, using the WHEN NOT MATCHED BY SOURCE clause of MERGE. Only one snapshot has been loaded so far, so every status_changed_date equals the snapshot date; the logic is in place for later snapshots.

3.2 Entity-relationship diagram and data dictionary

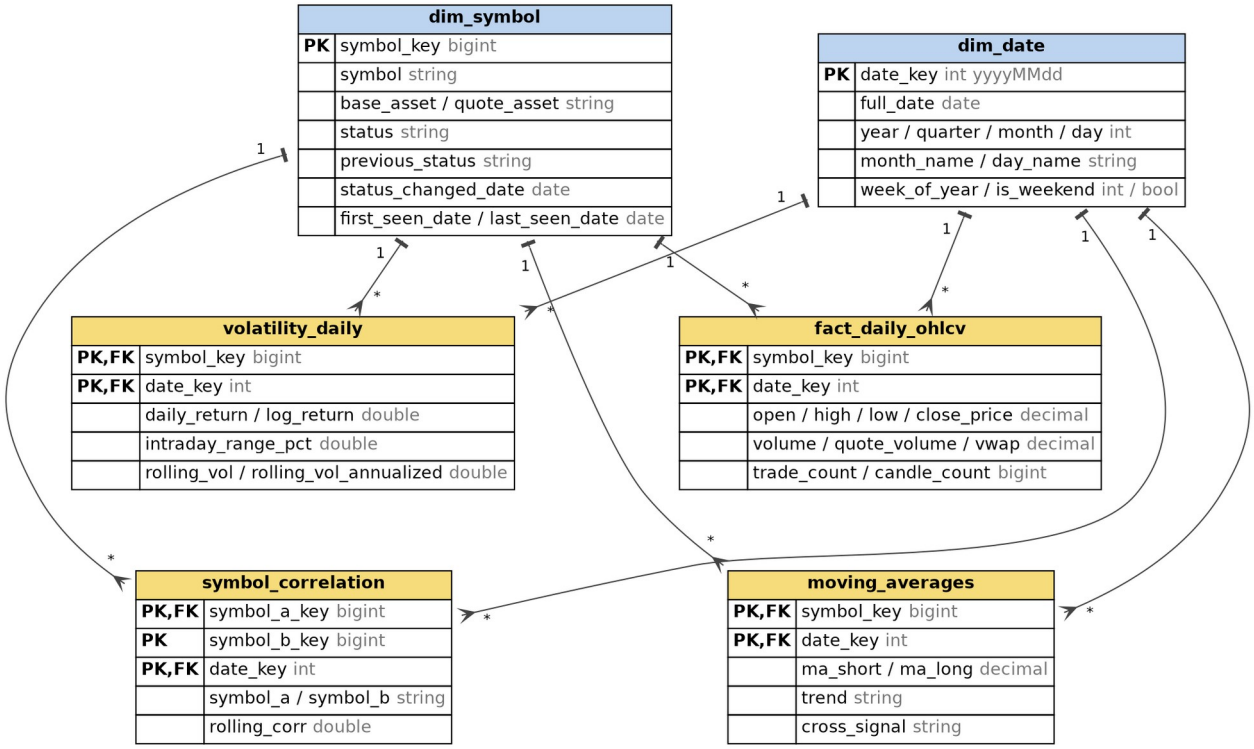


Figure 2. Gold ER diagram. Every fact table is many-to-one to dim_symbol and dim_date (symbol_correlation joins dim_symbol through symbol_a_key).

Column-level definitions, types, derivations and the business-question mapping are in docs/gold_data_dictionary.md in the repository. The table below summarises the measures.

Table	Measures and derivations
fact_daily_ohlcv	open = first candle open of the day (min_by on open_time); high = max; low = min; close = last candle close (max_by); volume and quote_volume = sums; vwap = quote_volume / volume (NULL when volume is 0); trade_count; candle_count (1,440 for a complete day).
volatility_daily	daily_return and log_return from consecutive closes; intraday_range_pct = (high - low) / open; rolling_vol = standard deviation of log returns over 7 days; rolling_vol_annualized = rolling_vol x sqrt(365).
moving_averages	Simple moving averages over 5 and 20 days; trend BULLISH or BEARISH from their order; cross_signal GOLDEN_CROSS or DEATH_CROSS on the day the order flips.
symbol_correlation	Rolling 14-day Pearson correlation of daily log returns for every pair among the top 15 symbols by quote volume; only rows with a full 14-day window are kept.

3.3 Read optimisation

After every load the notebook runs OPTIMIZE ... ZORDER BY on the four fact tables: (symbol_key, date_key) for fact_daily_ohlcv, volatility_daily and moving_averages, and (symbol_a_key, date_key) for symbol_correlation. Those are the columns the dashboard filters on (a symbol slicer and a date range). MERGE rewrites files, so OPTIMIZE also compacts the small files that repeated incremental merges leave behind. Each OPTIMIZE step is logged with its duration.

Table +

	version	timestamp	userId	userName	operati
1	12	2026-10-24T15:48:34.000+00:...	736135877272...	arqumumais@gmail.co...	OPTIMIZE
2	11	2026-10-24T15:47:48.000+00:...	736135877272...	arqumumais@gmail.co...	MERGE
3	10	2026-10-17T10:04:04.000+00:...	736135877272...	arqumumais@gmail.co...	OPTIMIZE
4	9	2026-10-17T10:03:40.000+00:...	736135877272...	arqumumais@gmail.co...	MERGE
5	8	2026-10-17T09:41:06.000+00:...	736135877272...	arqumumais@gmail.co...	OPTIMIZE
6	7	2026-10-17T09:40:34.000+00:...	736135877272...	arqumumais@gmail.co...	MERGE
7	6	2026-10-16T12:36:54.000+00:...	736135877272...	arqumumais@gmail.co...	OPTIMIZE
8	5	2026-10-16T12:36:29.000+00:...	736135877272...	arqumumais@gmail.co...	MERGE
9	4	2026-10-16T12:34:58.000+00:...	736135877272...	arqumumais@gmail.co...	OPTIMIZE
10	3	2026-10-16T12:34:34.000+00:...	736135877272...	arqumumais@gmail.co...	MERGE
11	2	2026-10-16T11:52:10.000+00:...	736135877272...	arqumumais@gmail.co...	OPTIMIZE
12	1	2026-10-16T11:51:45.000+00:...	736135877272...	arqumumais@gmail.co...	MERGE
13	0	2026-10-16T11:42:14.000+00:...	736135877272...	arqumumais@gmail.co...	CREATE TAE

Activate Windows
Go to Settings to activate Windo

13 rows 31.940s runtime Refreshed at 10/25/26, 10:27 AM (1m ago)

4. Pipeline integration and automation

4.1 Databricks Job

The Job cryptolens_daily chains the three notebooks as dependent serverless tasks. The Silver task waits for Bronze and the Gold task waits for Silver, so Gold refreshes automatically when a new incremental file has been processed into Silver.

Task	Notebook	Depends on	Parameters
bronze	01.2_bronze_ingest	(none)	load_type=incremental, interval=1m, year=2026, month=09, day=25
silver	02_silver_transform	bronze	load_type=incremental, interval=1m
gold	03_gold_transform	silver	(none: it chooses full or incremental mode from the state of the fact table)

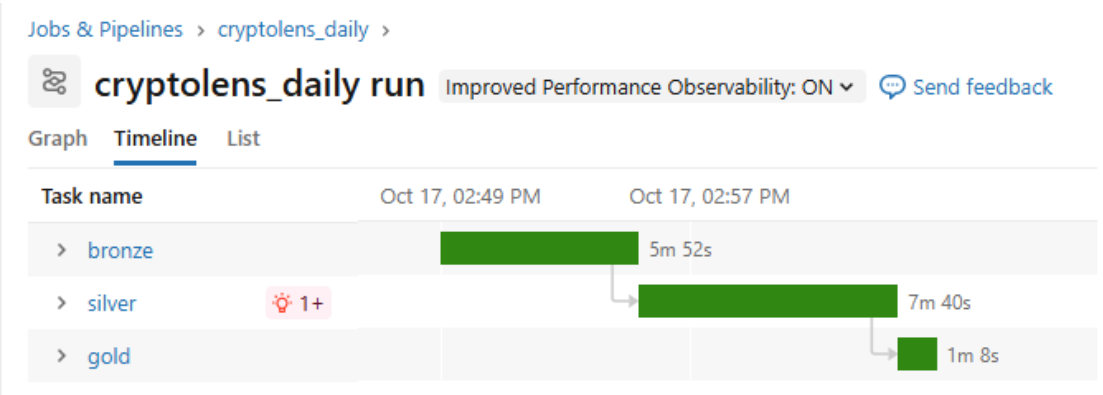


Figure 3. Timeline of one complete Job run: bronze 5 min 52 s, silver 7 min 40 s, gold 1 min 8 s (14 min 41 s in total).

The parameters are the widgets the notebooks already exposed, so no notebook is edited by hand to run another day. A daily trigger at 02:00 UTC is configured (after Binance publishes the previous day's files) and is currently paused, because the date parameters are fixed to 25 September 2026 and an active schedule would re-run the same day. Retries are switched off: every attempt writes rows to the audit table, and a silent retry would hide a real failure from the audit trail.

4.2 Incremental refresh and idempotency

The Gold notebook reads the latest trade_date already in fact_daily_ohlcv and reprocesses Silver from two days before it, then merges on (symbol_key, date_key). The two-day look-back absorbs corrected values. The table below shows the evidence collected during the project.

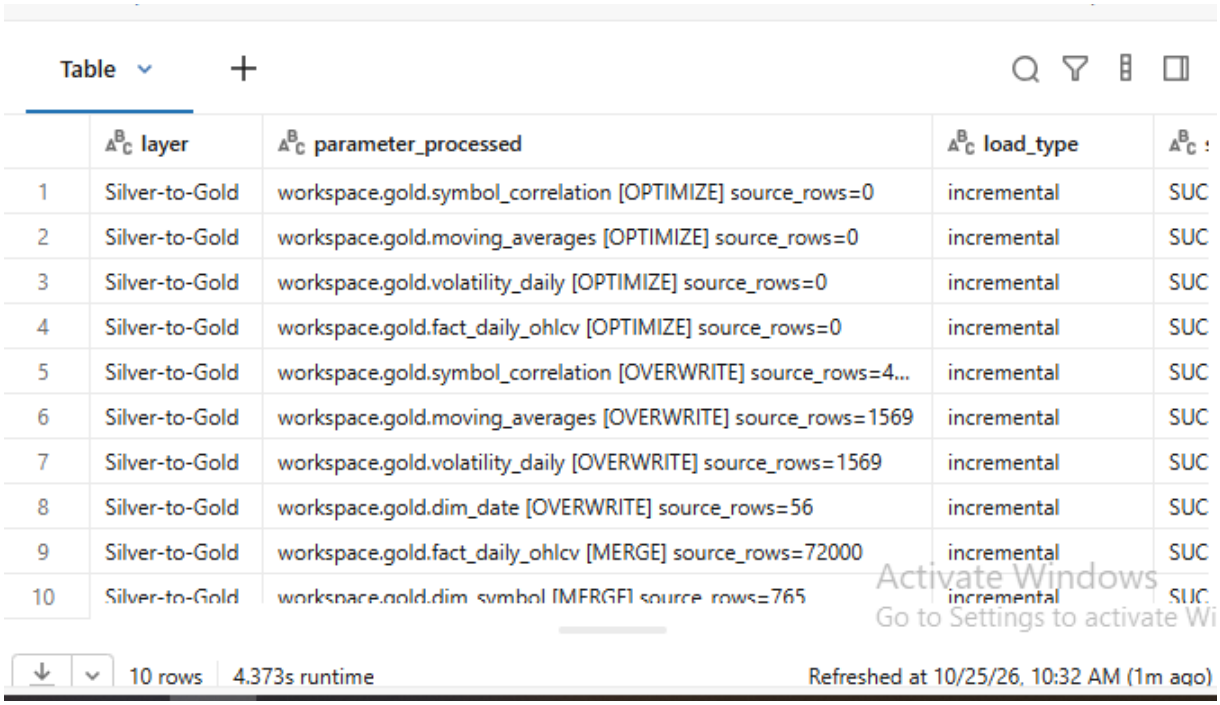
Run	What the notebook did	fact_daily_ohlcv rows after
Full build	Aggregated all August Silver candles	1,519 (49 symbols x 31 days)
Incremental mode, same data	Re-read the last 3 days (211,680 candles), merged 147 daily rows	1,519 (unchanged)
After the 25 September load reached Silver	Merged 50 new daily rows	1,569
Repeat run	Re-read 72,000 candles, merged the same 50 rows	1,569 (unchanged)



Figure 4. Verification output of a Gold run: row counts per table and zero orphan fact rows.

4.3 Logging continuation

Gold writes to the same pipeline_execution_logs table as Bronze and Silver, using the table's own schema, with layer = Silver-to-Gold. Each step (dim_symbol, fact_daily_ohlcv, dim_date, volatility_daily, moving_averages, symbol_correlation and the four OPTIMIZE steps) records a run_id, the step and the number of Silver rows read (in parameter_processed), the load type, start_time and end_time (so the execution time is their difference), the status (SUCCESS or FAILURE), rows_inserted, rows_updated and the error message. For MERGE steps the inserted and updated counts come from the Delta commit metrics (operationMetrics of DESCRIBE HISTORY), so they describe what MERGE actually did, not the size of the source.



Table

	layer	parameter_processed	load_type	
1	Silver-to-Gold	workspace.gold.symbol_correlation [OPTIMIZE] source_rows=0	incremental	SUC
2	Silver-to-Gold	workspace.gold.moving_averages [OPTIMIZE] source_rows=0	incremental	SUC
3	Silver-to-Gold	workspace.gold.volatility_daily [OPTIMIZE] source_rows=0	incremental	SUC
4	Silver-to-Gold	workspace.gold.fact_daily_ohlcv [OPTIMIZE] source_rows=0	incremental	SUC
5	Silver-to-Gold	workspace.gold.symbol_correlation [OVERWRITE] source_rows=4...	incremental	SUC
6	Silver-to-Gold	workspace.gold.moving_averages [OVERWRITE] source_rows=1569	incremental	SUC
7	Silver-to-Gold	workspace.gold.volatility_daily [OVERWRITE] source_rows=1569	incremental	SUC
8	Silver-to-Gold	workspace.gold.dim_date [OVERWRITE] source_rows=56	incremental	SUC
9	Silver-to-Gold	workspace.gold.fact_daily_ohlcv [MERGE] source_rows=72000	incremental	SUC
10	Silver-to-Gold	workspace.gold.dim_symbol [MERGE] source_rows=765	incremental	SUC

Activate Windows
Go to Settings to activate Wi

10 rows | 4.373s runtime | Refreshed at 10/25/26, 10:32 AM (1m ago)

5. Business intelligence dashboard

5.1 Connection

Power BI Desktop connects through the Databricks connector to the Serverless Starter Warehouse (2X-Small) with a personal access token, and imports the six Gold tables plus pipeline_execution_logs. Import mode was chosen over DirectQuery because the data is small (a few thousand rows), the report then responds instantly, and the warehouse does not have to wake up for every click. The trade-off is that the report shows data as of the last refresh.

5.2 Semantic layer

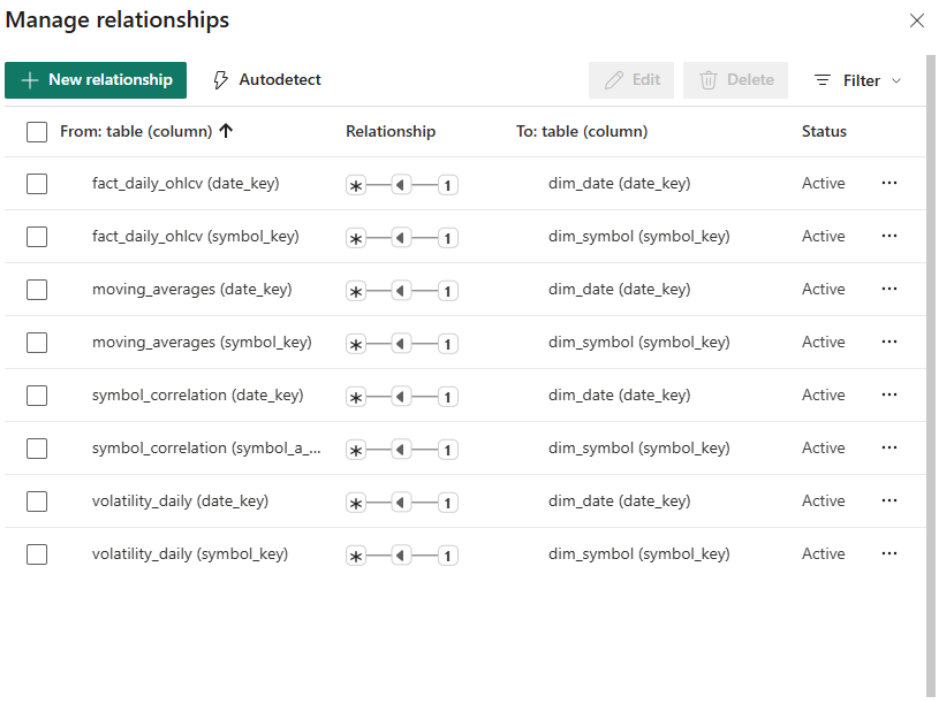
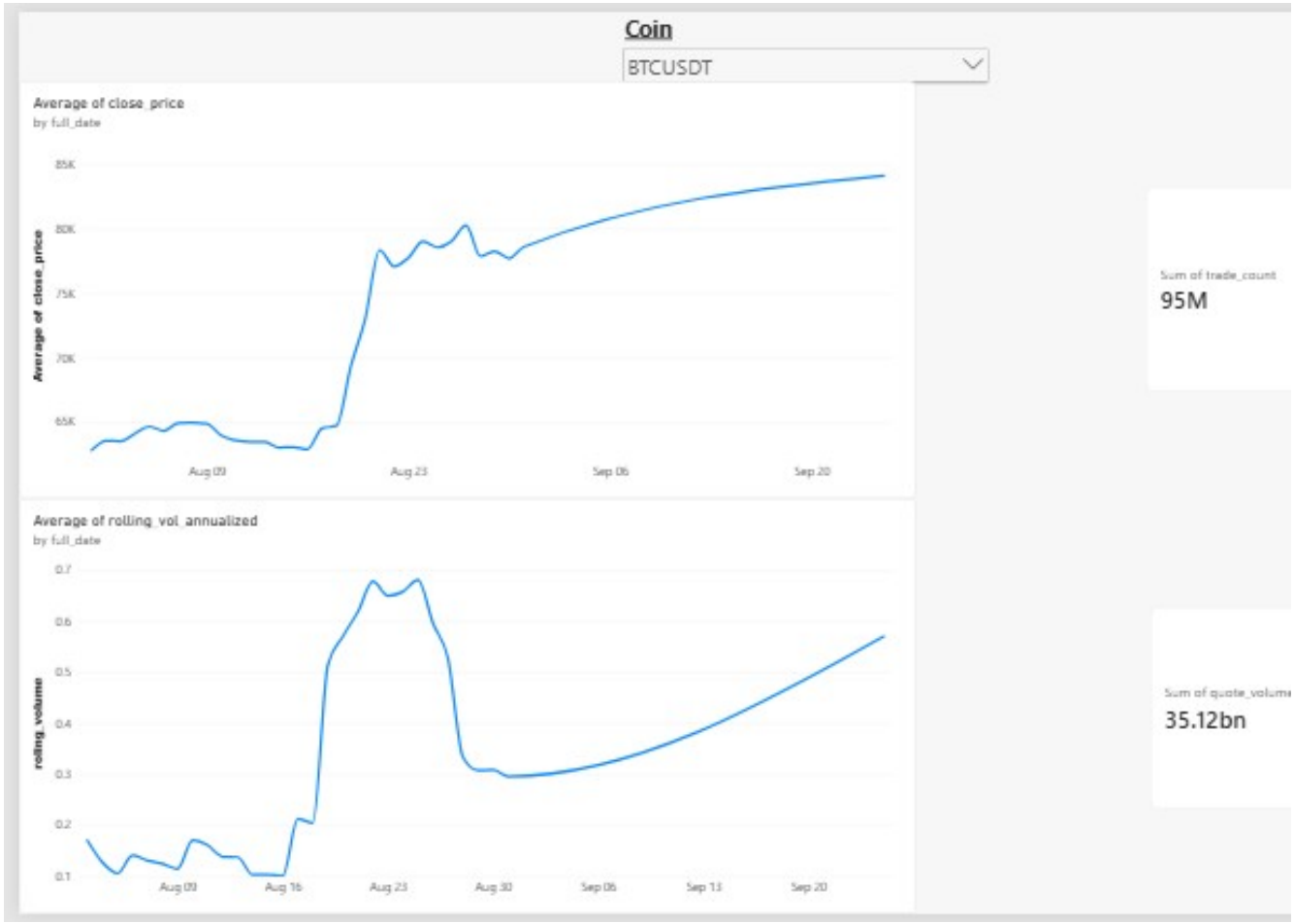


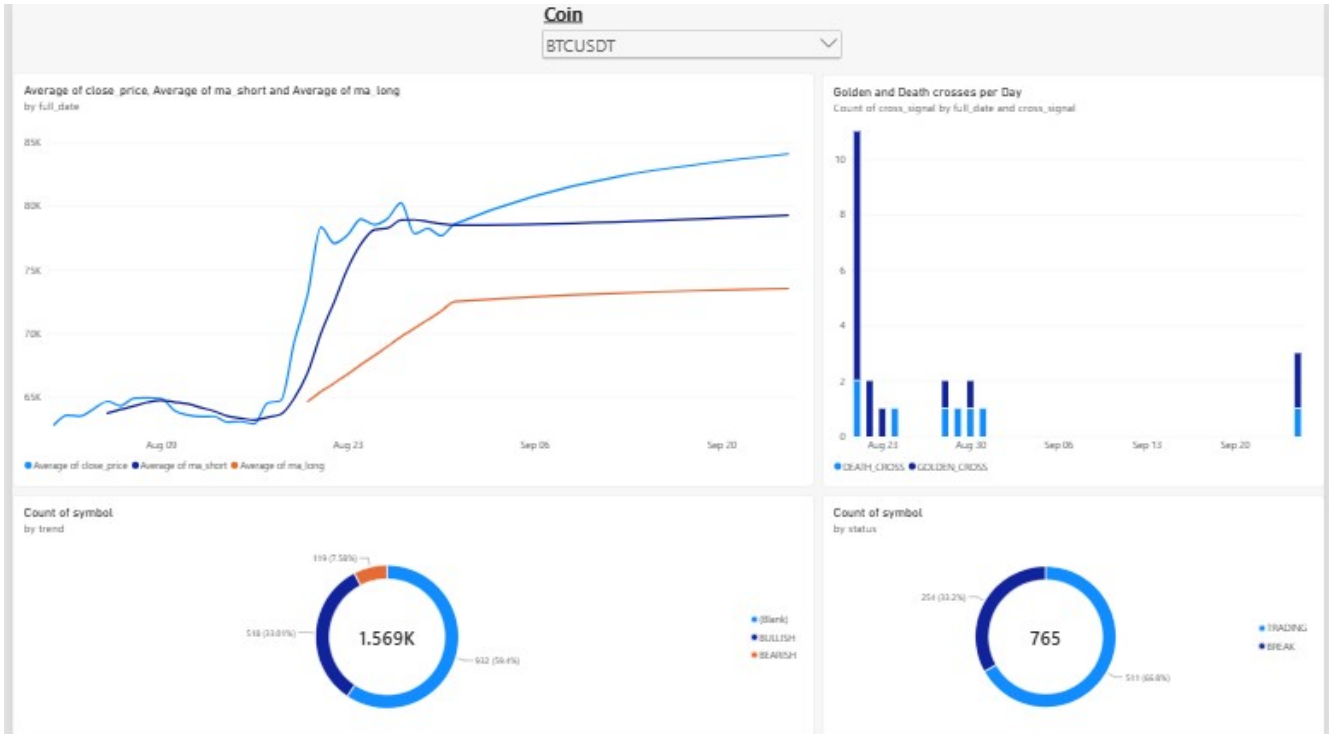
Figure 5. Power BI Manage relationships: eight active many-to-one relationships from the facts to dim_date and dim_symbol, single filter direction.

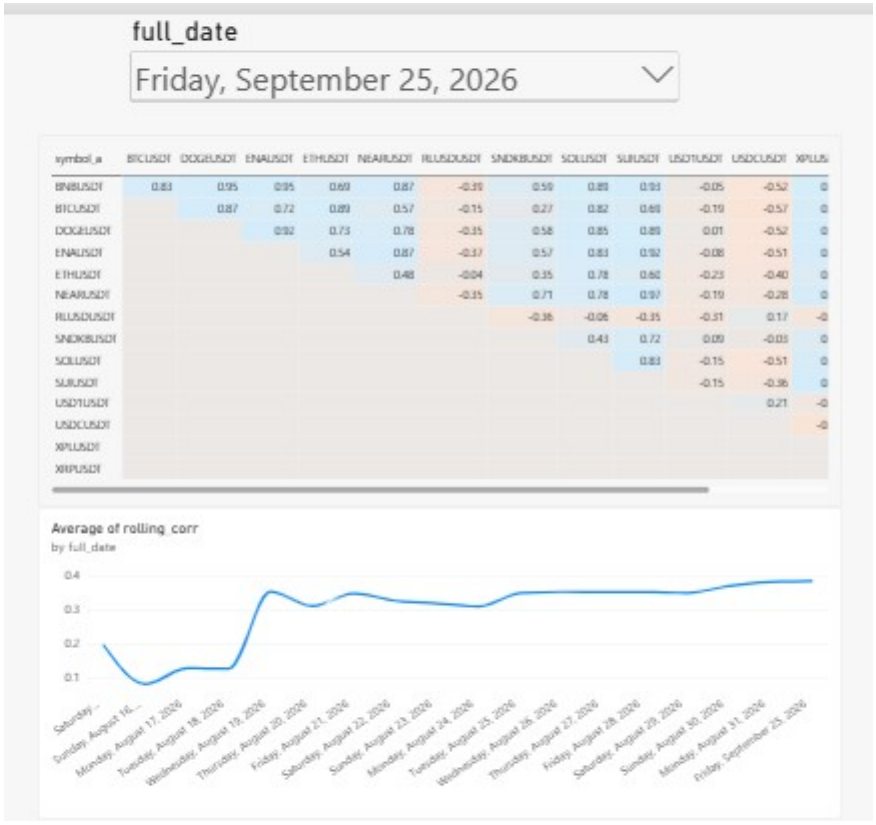
The model mirrors the star schema in Figure 2. The audit table stands alone, because it is not part of the dimensional model. The second link from symbol_correlation to dim_symbol (symbol_b_key) is deliberately not active: only one active path between two tables is allowed, and the heatmap uses the symbol_a and symbol_b text columns directly.

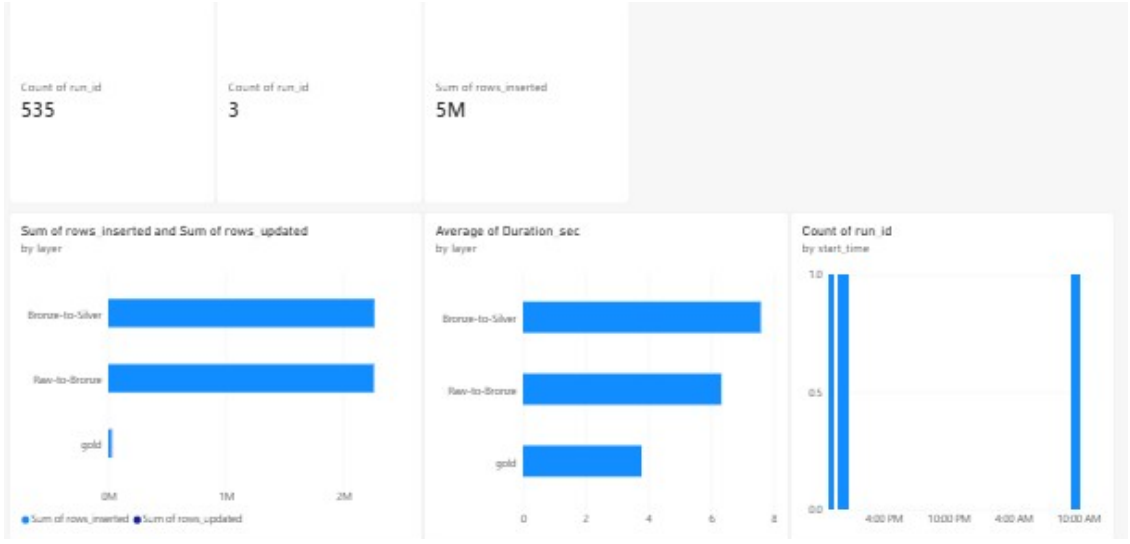
5.3 Pages and the business questions they answer

Page	Visuals	Business question
Price & Volatility	Coin slicer; close-price line; annualised-volatility line; KPI cards for total trades and USDT volume	How have price and volatility trended per asset?
Signals & Status	Close price with 5-day and 20-day averages; golden and death crosses per day; bullish vs bearish split; Binance USDT pairs by status	Which assets are bullish or bearish? Which pairs have changed status?
Correlation	Rolling-correlation matrix coloured from -1 to 1 for the top 15 symbols; average correlation over time	How correlated are the assets, and is it rising or falling?
Pipeline Audit	Log entries and failures; rows inserted and updated by layer; average step duration by layer; activity over time; error log	Is the pipeline healthy and what did each layer do?


Coin
BTCUSDT
Average of close_price
by full_date
Average of close_price
85K
80K
75K
70K
65K
Aug 09
Aug 23
Sep 06
Sep 20
Sum of trade_count
95M
Average of rolling_vol_annualized
by full_date
rolling_volume
0.7
0.6
0.5
0.4
0.3
0.2
0.1
Aug 09
Aug 16
Aug 23
Aug 30
Sep 06
Sep 13
Sep 20
Sum of quote_volume
35.12bn


Coin
BTCUSDT
Average of close_price, Average of ma_short and Average of ma_long
by full_date
85K
80K
75K
70K
65K
Aug 09
Aug 23
Sep 06
Sep 20
Average of close_price
Average of ma_short
Average of ma_long
Golden and Death crosses per Day
Count of cross_signal by full_date and cross_signal
10
8
6
4
2
0
Aug 23
Aug 30
Sep 06
Sep 13
Sep 20
DEATH_CROSS
GOLDEN_CROSS
Count of symbol
by trend
1.569K
(Blank)
BULLISH
BEARISH
Count of symbol
by status
765
TRADING
BREAK





5.4 What the dashboard shows

- BTCUSDT's average daily close rises from roughly 62,000 USDT in early August to roughly 84,000 USDT by 25 September, with a sharp jump in the third week of August; its annualised volatility peaks around the same time.
- Golden crosses cluster in the third week of August, when prices jumped; the dashboard flags them by day.
- The average pairwise correlation of the top-15 symbols rises from about 0.1 in mid-August to about 0.4 by 25 September. Stablecoin pairs (USDC, USD1) are near zero or negative, which pulls the average down.
- Of the 765 USDT pairs in the exchange snapshot, 511 (66.8%) are TRADING and 254 (33.2%) are BREAK. This chart covers the whole exchange, not only the 49 loaded symbols.

Reading caution. The data covers 1 to 31 August and 25 September only. Rolling windows are computed over rows, so values on 25 September treat it as the day after 31 August. Trend charts are limited to August to avoid drawing a line across the gap.

6. Decision log

6.1 Engineering decisions

#	Decision	Reasoning	Trade-off / alternative
1	Download locally, upload with the CLI	Databricks serverless could not reach the Binance endpoints. Downloading locally keeps the heavy transfer off billed compute and allows SHA256 verification before any file is trusted.	Adds a manual upload step; a production build would use an allow-listed network or a landing bucket.
2	Explicit StructType, never inferSchema	Inference costs an extra pass over the data and can change types between files; an explicit schema makes drift visible and keeps typing stable.	The schema must be updated when the source changes.
3	PERMISSIVE read plus a quarantine table	Rows that fail parsing are captured in bronze_quarantine instead of failing the batch, so one bad row does not stop the pipeline and nothing is lost silently.	Catches structural problems only (see 6.3).
4	MERGE instead of append at every layer	A re-run must not duplicate rows. Keys: Bronze (symbol, interval, load_type, open_time); Silver (symbol, interval, open_time); Gold fact (symbol_key, date_key).	MERGE rewrites matched files, so it costs more than an append.
5	Gold dimensions and facts share integer hash keys	One symbol_key and one date_key everywhere give clean one-to-many relationships and no lookups during the fact load.	Hash keys are not sequential; a collision is theoretically possible (none among 765 symbols).
6	Derived Gold tables overwritten from the fact table	Rolling indicators depend on previous rows, so recomputing from the fact table is simpler and always consistent. The fact table itself is merged.	Does not scale to hourly runs over years (see section 8).
7	try_divide for ratios	A zero-volume day raised DIVIDE_BY_ZERO under ANSI mode. NULL is the correct value for an undefined VWAP or return.	NULLs must be handled in the BI layer.
8	Indicator windows 5 / 20 days, volatility 7, correlation 14	About one month is loaded, so 50 / 200-day averages cannot be computed. Windows are variables in the config cell.	Short windows are noisier than the conventional ones.
9	Power BI Import mode	Small data, instant visuals, no warehouse wake-up per interaction.	Data is as fresh as the last refresh.
10	Job retries off, schedule paused	Retries would hide failures and add duplicate audit rows. The schedule stays paused until the date parameters are dynamic.	No automatic recovery from transient errors.

6.2 Partitioning

Gold. The Gold tables are not partitioned. The largest has 1,890 rows, which is kilobytes of data; partitioning by symbol (about 50 values) or by day would create many tiny files and slow every scan. Instead OPTIMIZE compacts each table and Z-ORDER clusters it on (symbol_key, date_key), the two columns the dashboard filters on.

Bronze and Silver. The combined klines data is about 2.26 million rows, a few hundred megabytes before compression. At that size partition pruning does not pay for itself, and partitioning by symbol or by day would produce many small files that need compaction.

6.3 Schema drift: the timestamp-unit incident

Stage	What happened
Assumption	The raw schema documents open_time and close_time as epoch milliseconds, and Silver converts them with a division by 1000.
Change in the source	Binance's 2026 kline files record the epoch in microseconds (16 digits instead of 13). The values still fit LongType, so they parsed without error.
Why nothing flagged it	Quarantine and PERMISSIVE mode catch structural drift (wrong type, malformed row, wrong column count). A change in the meaning of a valid number is invisible to them. Bronze and Silver row counts matched, so the verification notebook passed as well.
Detection	The first Gold build produced 31,000 distinct days (about 85 years) in dim_date, and the Silver timestamps fell in the year 58,000. A MIN/MAX query on open_time confirmed it.
Fix applied	Gold divides again by 1000 for any timestamp beyond the year 2200 and leaves genuine millisecond rows alone. Rebuilt Gold now covers 1 August to 25 September 2026 (56 dates, 1,569 fact rows).
Proper fix and prevention	Detect the unit in Silver from the magnitude of the raw value, and add a range check that sends implausible timestamps to quarantine. Until then Gold, not silver_klines, should be used for dates.

The pipeline also tolerates additive drift: Delta writes use mergeSchema, so a new column in a source file does not stop the load.

6.4 PII and governance

The dataset contains no personal data. Every field is an aggregated market statistic (prices, volumes, trade counts, trading-pair status); Binance publishes klines and exchangeInfo as public data. There are no usernames, account or wallet identifiers or IP addresses, and the audit table records only pipeline metadata. Controls applied anyway: Unity Catalog governs the tables and the Volume, personal access tokens are kept out of the repository and the notebooks, and bulk data folders and any .env file are excluded through .gitignore. One housekeeping point: the Volume path contains a team member's account email as a folder name. That is not dataset PII, but a production deployment would use a neutral folder and a service principal.

7. Spark UI and DAG analysis

The most complex Gold transformation is symbol_correlation: it reads two tables, ranks symbols, joins, runs a windowed aggregate over every pair and writes a Delta table. The notebook cell for this step was opened in the Spark UI (SQL / DataFrame tab) and its plan is shown below.

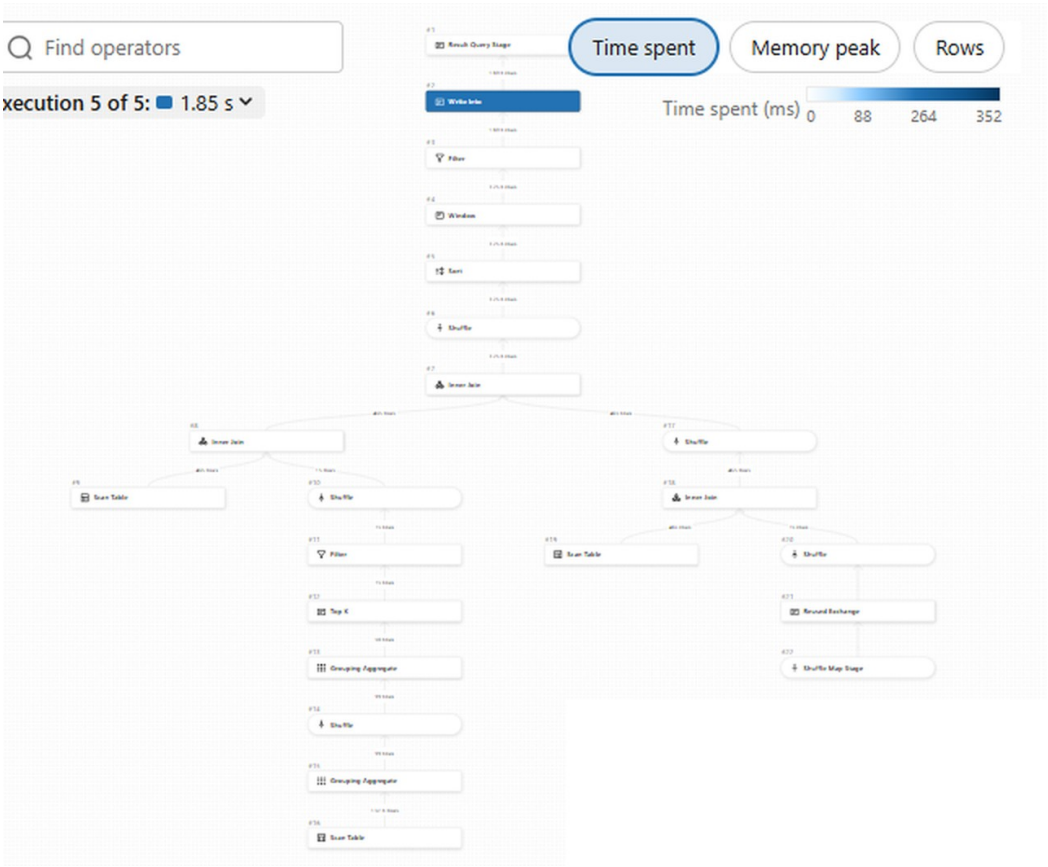


Figure 6. Databricks query profile (Spark plan graph) of the symbol_correlation write: scans, joins, two shuffles, window, sort, filter and Delta write. 223 KB read, 1,890 rows written.

7.1 How Spark executed the query

Plan stage	What Spark does	Concept
Scan fact_daily_ohlcv	Reads only symbol and quote_volume.	Column pruning: a columnar Delta/Parquet scan skips every other column.
HashAggregate, Exchange, HashAggregate, TakeOrderedAndProject	Sums quote_volume per symbol with a partial aggregate on each task, shuffles by symbol, finalises, then keeps the top 15.	Shuffle 1 (hash partitioning on symbol); map-side partial aggregation; sort plus limit fused so a full sort is avoided.
Scan volatility_daily with filter	The filter log_return IS NOT NULL is applied inside the scan.	Predicate pushdown (the scan node lists it under pushed filters).
Join with the 15 symbols	The 15-row result is sent to every task and joined without shuffling the large side.	Broadcast hash join chosen by the optimiser because one side is tiny.
Self-join on trade_date with symbol_a < symbol_b	Builds each unordered pair once per day (465 return rows, 15 symbols x 31 days, produce 105 pairs per day).	Equality join plus a residual condition; a broadcast join on this small input, or a shuffle on trade_date otherwise.
Exchange, Sort, Window	Redistributes rows by (symbol_a, symbol_b), sorts by trade_date inside each group and evaluates corr and count over ROWS BETWEEN 13 PRECEDING AND CURRENT ROW.	Shuffle 2 (hash partitioning on the window key) plus an in-partition sort; one task handles one pair.
Filter, Project, Write	Keeps rows with a full 14-day window, adds keys, and writes the Delta table in overwrite mode.	Delta transaction commit; the result is a single small file.

7.2 Observations

- Adaptive query execution coalesces the default 200 shuffle partitions into a few, because the data is a few hundred rows. Run time is dominated by scheduling and job start-up, not by compute or data movement. No skew or spill is expected at this size.
- The fact-table aggregation in section 4 (groupBy symbol and day over up to 2.2 million candles) is the largest data movement: a partial aggregate in each task, one shuffle on (symbol, trade_date), then a final aggregate.
- The incremental filter on trade_date in Gold is derived from open_time through the timestamp correction, so it is an expression, not a plain column comparison, and Delta file skipping probably cannot use it. Fixing the unit in Silver would let the date filter be pushed down to the scan.

8. Cost and FinOps evaluation

8.1 What was actually consumed

The project ran on Databricks Free Edition, which has no per-run billing and does not expose DBU consumption, and it did not use Azure for Students credits (the workspace is hosted on AWS). No cloud credits were consumed, so the figures below are a model built from measured run time and published list prices, not an invoice.

Measured run of cryptolens_daily	Duration	Share of run
bronze (50 daily files, MERGE per symbol)	5 min 52 s	40%
silver (MERGE per symbol)	7 min 40 s	52%
gold (whole-table operations, 10 logged steps)	1 min 8 s	8%
Total, one daily incremental batch	**14 min 41 s**	**100%**

An earlier run took 17 min 28 s because a second run overlapped it and shared compute; it is excluded. Bronze and Silver dominate because they loop over the symbols and run one MERGE per symbol, so the cost grows with the number of symbols, not with the number of rows.

8.2 Pricing assumptions

- Serverless Jobs compute: 0.35 USD per DBU (Premium plan, AWS US East list price, October 2026, as reported by third-party summaries of Databricks' pricing pages).
- Serverless SQL warehouse: 0.70 USD per DBU, same source and date.
- DBU rate per hour: not visible in Free Edition and not fixed for serverless jobs, so three assumptions are modelled: 2, 4 and 8 DBU per hour.
- The 2X-Small SQL warehouse used by Power BI is assumed to draw 4 DBU per hour; each refresh is assumed to run for 2 minutes.
- Storage: well under 1 GB of Delta data, so object storage is below 1 USD per year and is ignored. Serverless pricing includes the underlying machines.

Prices change; they must be re-checked on the Databricks pricing page before they are used for a real budget.

8.3 Daily versus hourly, annual compute cost

Cost = runs per year x run duration in hours x DBU per hour x 0.35 USD. The hourly case assumes that each run costs the same 14.7 minutes as the measured daily run, which is conservative for a smaller hourly increment but ignores growth in the number of symbols.

Scenario	Runs / year	Compute hours	2 DBU/h	4 DBU/h	8 DBU/h
Daily, as measured (14.7 min)	365	89	62 USD	125 USD	250 USD
Hourly, as measured (14.7 min)	8,760	2,141	1,499 USD	2,998 USD	5,996 USD
Hourly, optimised (4 min target)	8,760	584	409 USD	818 USD	1,635 USD

BI refresh cost. A SQL warehouse refreshed every hour (2 minutes at 4 DBU per hour and 0.70 USD) is about 292 hours and 818 USD per year; a daily refresh is about 12 hours and 34 USD. The refresh cadence of the dashboard is therefore as important as the pipeline cadence. The hourly enterprise scenario at 4 DBU per hour totals about 3,800 USD per year as measured (2,998 + 818) and about 1,640 USD optimised (818 + 818).

A realism check. Binance publishes the 1-minute kline files once per day, so a true hourly pipeline would have to read the REST API for the newest candles instead of the daily file. The measured 14.7 minutes would not apply as such, but it is the only measurement available and it sets the order of magnitude.

8.4 How the pipeline would be optimised

#	Optimisation	Expected effect
1	Replace the per-symbol loop in Bronze and Silver with one read of all files and one MERGE per layer	Targets the 92% of run time spent in the loops; the 4-minute figure in section 8.3 assumes this is achieved (a target, not a measurement).
2	Make the derived Gold tables incremental (recompute only the affected symbol-days and windows)	Removes whole-table rebuilds that grow with history.
3	Run OPTIMIZE weekly or use liquid clustering instead of optimising after every run	Removes four OPTIMIZE steps from every hourly run.
4	Fix the timestamp unit in Silver	Lets the date filter reach the scan, so incremental runs read only new files.
5	Match cadence to the source: daily for the file feed, a lightweight stream or API pull only if hourly freshness is needed	Avoids paying 24 runs a day for data that changes once a day.
6	Power BI refresh no more often than the data changes; keep the warehouse on auto-stop	Cuts the BI line of the cost table.
7	Use Jobs compute with spot instances or a smaller serverless size where the workload allows it, and monitor consumption with the system billing tables	Lower DBU price and visibility of real spend, which this project could not measure.

9. Known limitations

- Silver timestamps are wrong by a factor of 1000 (section 6.3); Gold compensates. Do not use silver_klines for dates.
- Only 1 to 31 August and 25 September 2026 are loaded, so row-based windows cross a gap, moving averages use 5 and 20 days, and one exchangeInfo snapshot means no status change has been observed yet.
- Bronze and Silver process one symbol at a time (section 8.1).
- The Job schedule is configured but paused, and its date parameters are fixed.
- Costs are modelled, not billed; DBU rates per hour are assumptions.

10. Reproducing the pipeline

1. Run the extractor notebook 01.1_bronze_ingest_data extractor.ipynb locally to create full_load, incremental_load and exchange_info (it downloads the files and verifies every SHA256 checksum).
2. Create the schema workspace.cryptolens and the volume bronze_raw; upload the three folders with databricks fs cp -r.
3. Run the Job cryptolens_daily, or run 01.2_bronze_ingest, 02_silver_transform and 03_gold_transform in order.
4. Check the counts in 00_layers_verification and in the last cell of 03_gold_transform.
5. Open dashboard/gold_layer_visualization.pbix, point the Databricks connector at your SQL warehouse and refresh.